

Feature Proposal: SecureAccess - Keycloak Integration for Centralized Authentication and Authorization

Feature Name

SecureAccess: Centralized Authentication and Role-Based Authorization with Keycloak

General Feature Description

What does the proposed feature do?

The proposed feature adds **Keycloak** as the identity and access management solution for the ss-web application. Instead of relying on locally implemented login logic and placeholder tokens, the application would delegate authentication to Keycloak through standard protocols such as **OpenID Connect (OIDC)** and **OAuth 2.0**.

In practical terms, this means that when a user wants to access protected areas of the platform, such as **Photos**, **Devices**, or **Statistics**, they will be redirected to a secure Keycloak login page. After successful authentication, Keycloak will issue signed tokens that the React frontend can use to call the Go backend, while the backend validates these tokens before allowing access to protected resources.

This feature can also support **role-based authorization**. For example:

- normal users can view only the pages and data relevant to their permissions;
- administrators can access device management operations and privileged actions;
- future operator or medical staff roles can be added without redesigning the entire authentication system.

How does it improve the application, and what benefits does it bring?

This feature improves the application in several important ways.

First, it transforms authentication from a temporary or incomplete mechanism into a **real production-ready security layer**. In the current project structure, the frontend still contains placeholder authentication logic, and the backend explicitly marks

authentication as not fully implemented. Because this application handles uploaded images, OCR-extracted content, device information, and medical-document-related statistics, weak authentication is a major risk.

Second, Keycloak introduces **centralized user management**. Instead of maintaining password policies, user sessions, token issuance, role mapping, and optional multi-factor authentication manually in the project code, these concerns are managed by a dedicated identity provider.

Third, the feature improves **scalability and maintainability**. As the project evolves and potentially includes a mobile client, more user roles, or integrations with institutional accounts, Keycloak provides a standard foundation that avoids repeatedly rewriting custom authentication logic.

Finally, it improves the **user experience** by offering a consistent login flow, secure session handling, and optional features such as password reset, e-mail verification, single sign-on, and multi-factor authentication.

Necessity Argumentation

Why is this feature useful for users?

This feature is useful because users of *ss-web* interact with data and functionality that should not be publicly accessible. The application is not a simple static website; it receives images from devices through MQTT, processes them with OCR, displays extracted content, and offers statistics that may include sensitive medical information. Users therefore need a trustworthy way to prove their identity before viewing or modifying data.

From the user perspective, Keycloak provides a more reliable and professional authentication experience. Instead of depending on a locally implemented login that may be incomplete or insecure, users benefit from:

- secure login with verified sessions;
- consistent access rules for each page;
- reduced risk of unauthorized access to images, reports, or device controls;
- future support for stronger security mechanisms such as MFA.

What problems does it solve, or what improvements does it introduce?

The feature solves several concrete problems already visible in the current architecture.

1. Incomplete authentication implementation

At the moment, the frontend authentication context contains development-oriented placeholder behavior, and protected routes are effectively bypassed. This means the application behaves as if users are already authenticated, even when no real verification has happened. Keycloak replaces this temporary model with a real authentication lifecycle.

2. Weak trust model for sensitive data

The platform stores and displays uploaded images, OCR results, and statistics derived from medical documents. Without strong authentication and authorization, unauthorized users could access information that should remain restricted. Keycloak ensures that access decisions are based on signed identity tokens and configured roles.

3. Difficult manual management of passwords and tokens

If the team continues with a custom solution, it must handle password hashing, reset flows, token generation, expiration, revocation, role assignment, brute-force protection, and auditability on its own. This is possible, but error-prone and time-consuming. Keycloak already provides these mechanisms in a mature platform.

4. Limited extensibility

The project already includes a web frontend, a Go backend, device communication through MQTT, and the possibility of future mobile extensions. A custom authentication system may work for the current lab version, but it becomes harder to extend when the number of users, roles, and clients increases. Keycloak is better suited for this growth.

How does it integrate into the application's usage flow?

The feature integrates naturally into the existing application flow.

1. A user opens the ss-web frontend.
2. If the user accesses a public page, the application behaves normally.
3. If the user tries to open a protected page such as /photos, /devices, or /statistics, the frontend checks whether a valid Keycloak session exists.
4. If no valid session exists, the user is redirected to the Keycloak login page.
5. After successful login, Keycloak redirects the user back to the application with valid OIDC tokens.
6. The React frontend uses the access token for API calls to the Go backend.
7. The Go backend validates the token signature and claims, then authorizes access based on roles.
8. If the user logs out, the session is terminated both in the application and in Keycloak.

This flow fits the current architecture very well because the application already separates concerns between frontend and backend. Keycloak becomes the authentication authority, while the Go server remains responsible for business logic such as photo access, device actions, statistics, and profile data.

Technical Justification

How could this feature be implemented?

The recommended implementation is to integrate Keycloak using the **Authorization Code Flow with PKCE**, which is the standard and secure approach for modern single-page applications.

On the frontend side, the React application can use Keycloak in one of two ways:

- through the official `keycloak-js` adapter;
- through an OIDC-compatible React library such as `react-oidc-context`.

The frontend would no longer use the current placeholder token model. Instead, it would initialize a Keycloak client at startup, detect whether the user already has an active session, and protect routes accordingly. The existing `AuthContext` can be refactored to expose:

- authenticated user state;
- access token;
- parsed roles;
- login and logout methods;
- token refresh handling.

On the backend side, the Go API would validate Keycloak-issued JWT access tokens before allowing access to protected endpoints. This can be implemented using:

- `coreos/go-oidc` for OIDC discovery and token verification;
- or `golang-jwt/jwt` together with Keycloak's **JWKS** public keys.

The server middleware would replace the current placeholder `noAuth` logic with a real `withAuth` middleware that:

- reads the `Authorization: Bearer <token>` header;
- verifies the token signature;
- checks token expiration and issuer;
- checks the intended audience or client ID;
- extracts claims such as `email`, `preferred_username`, and `realm_access.roles`;
- places identity and role information into the request context.

Keycloak itself can be deployed through **Docker Compose**, which matches the current project style. A new service can be added to the existing stack, alongside the React client, Go server, MongoDB, and MQTT broker.

What technologies, APIs, or algorithms could be used?

The feature can be implemented with the following core technologies:

- **Keycloak** as the identity provider and authorization server;
- **OpenID Connect** for authentication;
- **OAuth 2.0 Authorization Code Flow with PKCE** for secure login in the SPA;
- **JWT access tokens** signed with **RS256**;
- **JWKS** endpoint for public-key discovery and signature validation;
- **React + TypeScript** integration through `keycloak-js` or `OIDC` libraries;
- **Go middleware** for token validation and authorization checks;
- **Docker Compose** for local deployment.

Several additional Keycloak capabilities are also relevant:

- **realm roles** and **client roles** for role-based access control;
- **user federation** for future integration with institutional LDAP/Active Directory;
- **self-registration**, e-mail verification, and password reset;
- **MFA** through OTP or other second factors;
- **session management** and centralized logout.

What are the main technical challenges, and how can they be addressed?

Challenge 1: Replacing the current login/register flow

The current application already has login and register pages in the React client and local user logic in the Go server. Introducing Keycloak means the team must decide whether user registration remains local or moves entirely to Keycloak.

Solution: The cleanest option is to move authentication responsibilities to Keycloak and keep the backend focused on application data. If the application still needs local profile metadata, that data can be linked to the Keycloak user ID or e-mail after login.

Challenge 2: Token validation and role mapping in Go

Even if Keycloak issues tokens correctly, the backend must validate them consistently and interpret roles in the same way as the frontend.

Solution: Implement a shared authorization model. For example, `admin` can be required for device-control endpoints, while `user` can be sufficient for viewing photos or statistics. This mapping should be documented clearly in middleware and route configuration.

Challenge 3: Secure token handling in the browser

If tokens are stored insecurely, the application remains exposed to token theft through XSS or browser leaks.

Solution: Avoid long-term storage in `localStorage` when possible. Prefer short-lived access tokens, refresh-token rotation, and either in-memory session handling or a BFF-style secure-cookie approach if the architecture evolves. In addition, the frontend should enforce a strong Content Security Policy and safe rendering practices.

Challenge 4: Development and deployment complexity

Adding Keycloak introduces configuration overhead: realms, clients, redirect URIs, roles, secrets, and environment variables.

Solution: Version the Keycloak configuration for development, document the setup in the repository, and use Docker Compose to keep local startup reproducible. This makes the feature manageable even for a student project.

Impact on Performance and Security

How does this feature affect the application's performance?

The performance impact is generally acceptable and, in many cases, beneficial compared with a custom authentication solution.

During login, users will experience one extra redirection to Keycloak, but this cost is small and happens only when a new session is needed. After authentication, the backend can validate JWT tokens locally using Keycloak's public keys, which is fast and does not require a database query for every request.

There are still several performance considerations:

- Keycloak itself becomes an additional service that consumes CPU and memory;
- token refresh operations create periodic network traffic;
- initial OIDC discovery and public-key retrieval add a small startup overhead.

These effects can be mitigated through:

- caching Keycloak metadata and JWKS keys;

- using short, efficient middleware for token verification;
- deploying Keycloak in Docker with appropriate resource limits;
- avoiding token introspection on every request unless strictly necessary.

Overall, the feature should not significantly slow down the application, and it may even reduce backend complexity by removing custom authentication logic and repeated database checks for credentials.

Are there any security risks associated with it? If so, how can they be mitigated?

Yes. Although Keycloak greatly improves security, integration mistakes can still introduce vulnerabilities.

Risk 1: Misconfigured redirect URIs

If redirect URIs are too permissive, attackers may abuse the login flow or intercept tokens.

Mitigation: Configure only explicit trusted redirect URIs and origins for the frontend.

Risk 2: Token theft in the frontend

If access or refresh tokens are stored unsafely, an attacker who exploits XSS may steal them.

Mitigation: Prefer in-memory tokens or secure cookies, keep token lifetime short, sanitize UI rendering, and enforce CSP headers.

Risk 3: Incorrect role enforcement

If the frontend hides buttons but the backend does not enforce roles, unauthorized users may still call privileged endpoints directly.

Mitigation: Treat backend authorization as the real enforcement layer. Every sensitive endpoint, especially device-control or administrative endpoints, must verify roles server-side.

Risk 4: Trusting tokens without validating issuer, audience, and expiration

Accepting any signed token would be a serious security flaw.

Mitigation: The Go middleware must validate signature, issuer, audience, expiration time, and intended client.

Risk 5: Centralized dependency on Keycloak

If Keycloak becomes unavailable, users may be unable to log in.

Mitigation: Use proper container health checks, persistent storage, backups for realm configuration, and clear operational documentation.

In conclusion, the security benefits clearly outweigh the risks. For an application that processes sensitive uploaded documents and exposes privileged operations such as device management, Keycloak is not only a useful feature, but a strategically important one.

Conclusion

The integration of Keycloak into *ss-web* is a strong and realistic feature proposal because it addresses a current weakness in the project while also preparing the application for future growth. It improves authentication quality, enables role-based access control, reduces the need for custom security code, and aligns the system with modern web security standards.

Because *ss-web* already has protected pages, user flows, and backend endpoints that expect authenticated access, Keycloak fits naturally into the existing architecture. For a project that handles image uploads, OCR-processed content, device management, and medical-document statistics, this feature provides both immediate practical value and long-term architectural benefits.